1. 单处人工故障校准

开发集人工故障 · 检查器改进

数据从哪里来：五个已有开发题的正确参考 IR：序列检测、粘滞报警、奇偶状态、延迟请求与去抖。对状态转移、输出或复位规则逐次只改一处；排除语义等价变体，留下 163 个错误变体。

怎样修改 / 测试：正确参考 → 单处编辑 → 精确有限状态图确认有错 → 旧公开检查 → 为漏检构造反例 → 独立程序参考核对 → 贪心选出 12 个新增公开测试 → 固定检查复测。另生成对应人工 RTL，检查它是否忠实实现所改 IR。

为什么这样设计：先故意制造“答案已知的错误”，测量检查器的盲点。用 25 个组合校准“规格→IR”与“IR→RTL”等可观察错误关系，避免把所有失败混在一起。

报警一旦置位，应保持到复位

实际人工变体：sticky_alarm__edge_05_S0

上升沿后采样	第 1 拍	第 2 拍	第 3 拍
输入	1	1	1
正确输出	0	1	1
错误输出	0	1	**0**

只改一条转移：报警态收到 1 时，错误返回初态。红格是首次分歧。

实际结果：旧公开检出 133/163（81.6%）；增强公开检出 163/163。五题族新增测试数为 8、1、0、1、2，总计 12。25/25 诊断组合符合预期。原固定隐藏测试也检出这 163 个故障。

结论边界：12 个新测试来自这批开发故障，因此 100% 是开发集覆盖率，不是未知错误检出率或模型成功率。人工参考与判据提供交叉核对，但不是独立外部团队认证。

证据索引（均相对于 outputs/rtl-batch；图是原始记录的注释重绘，不是工具截图）

MUTATION_FINDINGS.md

evidence/mutation-v1/cases.json

evidence/mutation-v1/summary.json

2. 双处编辑故障探索

冻结检查器复测 · 受限组合故障

数据从哪里来：仍使用上述五个开发题；按限定编辑规则同时改变两个不同字段，得到 3,119 个不等价 IR 变体。这些是少量题目的派生错误，不是 3,119 个独立设计。

怎样修改 / 测试：正确 IR → 枚举两处编辑 → 排除等价情况 → 用已冻结的旧/增强公开检查比较 → 每题按枚举顺序取前 10 个故障做 RTL 检查，共 50 个。另查正确实现与不改变语义的改写。

为什么这样设计：两处改动可能互相掩盖，不能由“单处全检出”直接推断组合错误也能检出。本轮使用上一轮冻结的增强判据，不根据新结果继续调参。

两条转移一起改，旧公开测试仍放过

实际变体：rt_sequence__edge_00_S1__edge_03_S0

修改位置 / 观察	正确版本	错误版本
S0 收到 0	S0	**S1**
S1 收到 1	S1	**S0**
输入 0001 后输出	0000	**0001**

此错误变体：旧公开漏检，增强公开检出。整轮 3,119 个 IR 故障中，50 个做 RTL 仿真。

实际结果：IR 层：旧公开漏 176/3,119，增强公开漏 0。50 个 RTL 故障候选均被隐藏仿真检出，并均与其错误 IR 行为一致。

结论边界：人工、受限、开发题上的结果。RTL 子集按枚举顺序选择，并非随机代表样本；不能把 IR 全量结果表述为 3,119 次 RTL 仿真。

证据索引（均相对于 outputs/rtl-batch；图是原始记录的注释重绘，不是工具截图）

exploration/20261011/RESULTS.md

exploration/20261011/runs/compound-v1/cases.json

exploration/20261011/runs/compound-v1/simulation-results.json

3. 延迟触发故障

人工构造固定测试盲区 · 有界求解

数据从哪里来：五个开发题分别加入匹配特定 12 位序列的额外状态；前缀偏离触发词时转回正常逻辑。它们是刻意避开已有固定测试的错误，不是模型自然输出。

怎样修改 / 测试：选择触发词 → 构造额外前缀状态 → 核对所有长度 0–11 的输入 → 运行旧公开/隐藏测试 → 添加触发输入见证 → 用有界 SAT 搜索并检查反例。

为什么这样设计：固定试题只抽查有限输入。SAT 则在给定范围内寻找“能让参考与实现不一致的输入”；把固定测试通过和更强正确性证据区分开。

前 11 位都一样，第 12 位才暴露

实际 rt_sequence 人工延迟触发样本；以下为复位后各输入采样位置

位置	1	2	3	4	5	6	7	8	9	10	11	12
输入	1	1	1	1	1	0	0	1	1	0	0	1
正确	0	0	0	0	0	0	0	1	0	0	0	1
错误	0	0	0	0	0	0	0	1	0	0	0	0

固定测试漏检；新见证和有界 SAT 检出。SAT 的步号另含初始化/复位，不能混作位序。

实际结果：五个案例在旧公开及旧隐藏测试中全漏检；新见证与 SAT 均检出 5/5。每题长度 0–11 共 4,095 条输入保持一致，五题合计 20,475 条。SAT 使用界限 14。

结论边界：故障被专门设计为躲开旧测试，不能用 5/5 推算 AI 自然错误频率。图中的第 12 个输入位置与 SAT 的初始化/复位步号不是同一计数。

证据索引（均相对于 outputs/rtl-batch；图是原始记录的注释重绘，不是工具截图）

exploration/20261011/RESULTS.md

exploration/20261011/runs/delayed-v1/cases.json

exploration/20261011/runs/delayed-v1/summary.json

exploration/20261011/runs/formal-v1/summary.json

4. R1b：历史 RTL 有界检查

历史真实模型输出 · 本轮零模型调用

数据从哪里来：从历史 fsm48、paired-a32 与 paired-b64 输出中选出 24 份 RTL：原固定测试 17 份通过、7 份失败。保留原输出字节与来源哈希，不修改模型答案。

怎样修改 / 测试：历史 RTL 与参考 → 构造输出差异检查电路 → Yosys 在 14 与 32 两个界限求解 → 保存反例输入 → Icarus 独立回放 → 核对实际不符与回执。

为什么这样设计：求解器报告的反例也需要仿真复现。双工具交叉验证可以排查转换和判题错误，但同一参考规格仍可能带来共同盲区。

历史模型 RTL：求解器找输入，仿真再确认

真实历史样本：fsm48 / m02__rt_sequence__prose__s001；未修改模型 RTL

证据步骤	这份代码的实际记录
旧判题	FUNCTION_FAIL
SAT 界限 14	找到反例；solver mismatch step = 10
交给 Icarus 的输入	先复位，然后 01001001
独立回放	18 次检查，1 个不符
另一界限 32	同一份代码也找到反例

整轮是 7 份已知失败 × 2 个界限 = 14 条反例；17 份原通过未发现新反例。

实际结果：7 份已知失败在两个界限都产生反例，共 14 条，全部仿真复现。17 份原通过没有发现新反例。本轮新增自然错误发现数为 0。

结论边界：“未发现反例”只限所用建模与界限；不代表无限时域正确。14 条反例来自 7 份失败代码，不能当成 14 个独立失败设计。

证据索引（均相对于 outputs/rtl-batch；图是原始记录的注释重绘，不是工具截图）

exploration/20261011/R1B_RESULTS.md

exploration/20261011/runs/r1b-v2/history-results.json

exploration/20261011/runs/r1b-v2/history/00_rt_sequence/14/witness/verdict.txt

5. R1c：事件与复位复测

历史 RTL 复查 + 人工时序校准

数据从哪里来：候选池有 99 条历史 PASS 记录、78 个不同 RTL 字节哈希；按题族/模型分配选出其中 64 份，14 份未选。覆盖五个题族。图示则是用来校准检查器的人工同步复位错误，不能计为历史代码新错误。

怎样修改 / 测试：先用正确/错误 RTL 校准事件判据 → 枚举输入前缀、时钟高低电平、复位脉冲与输入组合 → 独立程序参考和状态图交叉核对期望 → 对 64 份历史 RTL 实际仿真。

为什么这样设计：只在时钟沿检查容易漏掉异步复位、无时钟沿保持和输入泄漏问题。补充实验先正确初始化，再单独制造目标事件，避免把初始 X 误当作目标故障证据。

没有新时钟沿，异步复位也应该清零

人工校准样本 sync_mid；先正确初始化，再在中途拉低 rst_n

事件编号	3	4	5	6
clk	1	0	0	0
rst_n（0 为复位）	1	1	0	1
参考输出	1	1	0	0
错误同步复位输出	1	1	**1**	**1**

事件 5：clk 保持 0，rst_n 从 1 变 0；应输出 0，实际仍为 1。

实际结果：64/64 历史 RTL 通过，新增自然错误 0。五题族各 90,972 个参考事件，合计 454,860 次独立期望核对。图示人工错误在事件 5、6 被检出。

结论边界：纠正旧简称：64 是去重选出的 RTL 文件数，不只是 64 个事件配置；454,860 是参考事件核对数，不是 64 次仿真事件总和。数字仿真不覆盖物理亚稳态或真实 setup/hold。

证据索引（均相对于 outputs/rtl-batch；图是原始记录的注释重绘，不是工具截图）

exploration/20261011/R1C_RESULTS.md

exploration/20261011/runs/timed-reset-v1/selection.json

exploration/20261011/runs/timed-calibration-v2/plan.json

exploration/20261011/runs/timed-calibration-v2/sync_mid/simulate.stdout.log

6. 握手判据 v1

人工轨迹与手写 RTL · 事务交付判据

数据从哪里来：22 条人工编排轨迹，以及 10 个实际编译仿真的手写 RTL 案例。场景来自有限广播/握手开发契约，含合法延迟、停顿、丢失、重复、损坏及无效输入。

怎样修改 / 测试：先声明输入计划和合法规则 → 采集 valid/ready/data/last → 在 valid 与 ready 同时为 1 的采样点记一次交付 → 每个接收方独立维护待交付队列 → 检查顺序、内容、稳定性和有限排空。

为什么这样设计：像逐个登记包裹：接收方暂时不能收时，发送方要保持包裹不变。合法电路允许不同延迟，判据不应强求所有波形逐拍等于某个实现。

接收方还没准备好，数据不能偷偷变

真实执行的手写 RTL：stall_data_break，观察 consumer0

周期	4	5	6	7	8	9
valid	1	1	1	1	1	1
ready	0	0	0	0	0	1
data	17	**16**	**17**	**16**	**17**	**16**

周期 4 开始等待；周期 5 起反复改变数据，违反稳定性。周期 9 才发生交付。

实际结果：22 条轨迹预期分类全部匹配。10 个真实执行的手写 RTL 中，3 个合法实现通过，7 个人工故障检出。图示周期 5 已出现等待中数据改变。

结论边界：实际运行 RTL 工具，不等于错误来自 AI。开发契约、参考与检查器仍有同源风险；有限排空通过不代表任意停顿长度下都正确。

证据索引（均相对于 outputs/rtl-batch；图是原始记录的注释重绘，不是工具截图）

exploration/20261011/handshake-dev-v1/RESULTS.md

exploration/20261011/handshake-dev-v1/CONTRACT.md

exploration/20261011/handshake-dev-v1/calibration-v1/rtl/stall_data_break/normalized-trace.json

exploration/20261011/handshake-dev-v1/calibration-v1/rtl/stall_data_break/verdict.json

7. 握手判据 v2

保留负结果 · 复位取消与握手因果

数据从哪里来：新增有限 8 条手推轨迹和 8 个手写 RTL 案例，围绕中途复位、接受前提前输出、另一种合法双队列缓冲结构。保留 v1 原证据。

怎样修改 / 测试：冻结复位取消规则及预期类别 → 同时安排合法结构与人工故障 → 记录 80 周期轨迹 → 按复位划分 epoch → 逐项核对接受、取消、交付，以及完整错误类别。

为什么这样设计：“先送出、后接受”即使最终数据序列相同，也违反因果关系。复位则切断新旧事务：被取消的旧数据不能在复位后重新冒出。

复位取消了旧包裹，旧包裹却又被送出

实际手写 RTL：shared_reset_stale；数值用十进制表示

周期	输入 / 消费事件	判据含义
3	接受旧数据 17	旧 epoch 的事务
4	consumer0 收到 17	consumer1 尚未收到
8–9	复位；取消 consumer1 待交付 17	旧事务不应跨复位存活
10	**consumer1 却收到旧数据 17**	**UNEXPECTED_TRANSACTION**
11	才接受新数据 161	不能为周期 10 的旧输出作解释

故障抓到了；实际分类与预声明不一致，因此完整类别校准仍是 15/16。

实际结果：通过/失败状态 16/16 与预期一致；完整错误类别 15/16 一致。图示本应失败且确实失败，但原预声明 DATA_OR_ORDER_FAIL，实际为 UNEXPECTED_TRANSACTION；校准失败状态保持。

结论边界：抓到错误与准确命名错误是两个指标，不能只展示 16/16。合法提前 valid 只在该固定 ready 场景验证，不能泛化为任意组合握手都安全。

证据索引（均相对于 outputs/rtl-batch；图是原始记录的注释重绘，不是工具截图）

exploration/20261011/handshake-dev-v2/RESULTS.md

exploration/20261011/handshake-dev-v2/calibration-v2/summary.json

exploration/20261011/handshake-dev-v2/calibration-v2/rtl/shared_reset_stale/verdict.json

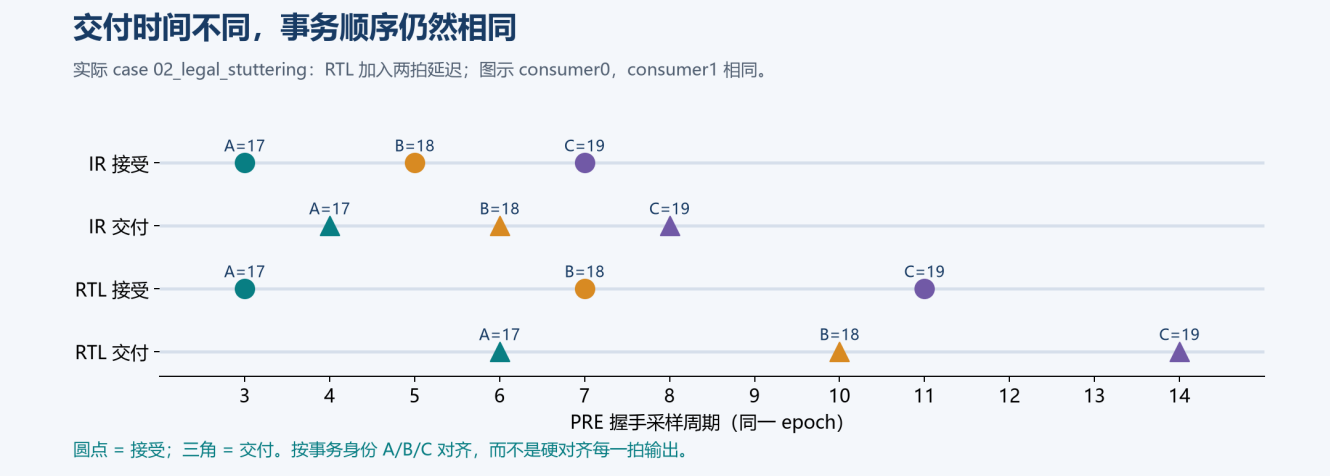
8. 事务域 IR / RTL 对齐

有限人工映射 · 三路判断

数据从哪里来：以当前 R6 寄存器表达式 IR 建立人工广播映射，再与手写 RTL 配对。预声明 12 个场景：11 个执行，1 个因复位规则含糊而事先排除。它们尚非新外部保留任务。

怎样修改 / 测试：冻结 PRE/POST 采样、复位 epoch 与输入身份 → IR 和 RTL 各运行自己的合法输入驱动 → 先分别检查是否满足规格 → 再按事务身份及取消集合比较 → 输出通过、失败或不可评估。

为什么这样设计：比较的是同一事务有没有正确到达，而非每一拍是否一样。若复位前双方接受的输入集合不同，直接比较最终序列可能冤枉合法实现，需要明确“不具备可比条件”。



实际结果：11 个执行案例的意图比较：6 通过、2 人工故障检出、3 不可评估。3 个原因分别是接受/取消集合不同、输入违约、采样缺失。另 1 项未执行。图示合法不同延迟仍通过。

结论边界：只验证有限映射与固定场景，不能宣称整个 IR 语言与所有 RTL 等价。未证明复杂外部任务上的自然错误定位价值；不把不可评估计作 RTL 故障。

证据索引（均相对于 outputs/rtl-batch；图是原始记录的注释重绘，不是工具截图）

exploration/20261015/transaction-alignment-v1-report/RESULTS.md

exploration/20261015/transaction-alignment-v1/run-v1/summary.json

exploration/20261015/transaction-alignment-v1/run-v1/02_legal_stuttering/ir-assessment.json

exploration/20261015/transaction-alignment-v1/run-v1/02_legal_stuttering/rtl-assessment.json